

Summary of process calculus

In **ω_0 -calculus**, a system description consists of a set of nodes, each running a sequential process annotated by its interface. Here's an overview of the key components:

- **N** and **P** denote the sets of all nodes and all processes, respectively. Nodes are represented by **M** and **N**, while processes are represented by **P** and **Q**.
- Names are drawn from two disjoint sets: **Pn** and **Gn**.
 - **Pn** contains **process names** (pnames), which are used for data values.
 - **Gn** contains **group names** (gnames), which are used for process interfaces (i.e., the communication groups that a process can belong to).
- **x**, **y**, and **z** represent process names, while **g** (possibly subscripted) represents group names.

The **ω_0 -calculus** defines a two-level syntax:

1. **Node level**: Nodes are the entities that execute processes. A node's behavior depends on its interface, which specifies the groups it can communicate with.
2. **Process level**: Processes perform computations and communications. Processes exchange data using process names and define their communication range through group names (which indicate which other nodes they can communicate with).

The syntax of **ω_0 -calculus** processes is defined by the following grammar:

$$P ::= \text{nil} \mid \text{Act}.P \mid P + P \mid [x = y]P \mid A(\text{xbar})$$
$$\text{Act} ::= \text{bxbar} \mid r(x) \mid \tau$$

In **ω_0 -calculus**, the key actions and process behavior are defined as follows:

- **Local broadcast** of a value **x** is represented by the action **bx**, meaning a process broadcasts the value **x** to all other processes within its communication range.
- **Reception** of a locally broadcasted value **x** is represented by **r(x)**, meaning a

process receives the value x .

- **Internal (silent) actions** are denoted by τ , representing actions that are not observable externally.
- **Process nil** represents the **deadlocked process**, which can perform no further actions.
- **Act.P** represents a process that can first perform the action Act and then behave as P.
- **The operator +** is used for **nondeterministic choice** between actions, meaning the process can choose between multiple actions nondeterministically.
- **[x = y]P** is a **matching construct** that behaves as process P if x and y (both process names) are equal. If x and y do not match, the process behaves as nil (deadlocked).
- **A($\rightarrow x$)** denotes **process invocation**, where A is a process identifier (i.e., a function name) and $\rightarrow x$ is a list of actual parameters (process names) being passed to it.
- A **process definition** is written as $A(\rightarrow x) \text{ def } = P$, which associates the process identifier A with a specific process expression P, where $\rightarrow x$ is a list of formal parameters (distinct process names) passed to P. These definitions can be **recursive**, allowing a process to invoke itself within its definition.

ω_0 -calculus node expression:

$M ::= 0 \mid P : G \mid (\nu g)M \mid M \mid M$

Here's what the symbols mean:

1. **0**: This represents an **inactive node**, meaning a node that has no process or behavior, essentially a dead or terminated node.
2. **P : G**: This represents a node with a process **P** and an interface **G**. Here, **G** is a set of group names (gnames) that define the communication capability of the node. **P** is the process executed by the node, while **G** defines which other nodes it can communicate with.
3. **(νg)M**: This operator is used to restrict the scope of the group name **g**. It limits the visibility of **g** to within the expression **M**, much like a local variable in

programming, meaning **g** is confined to this scope and cannot be accessed from outside.

4. **M | M**: This denotes the **parallel composition** of two node expressions. It means that the two nodes exist and run concurrently, with each node performing its process independently. The nodes may communicate or interact with each other through their interfaces **G**.

Explanation:

- **P : G** is called a **basic node expression** because it defines the basic structure of a node: a process and its interface.
- **(vg)M** and **M | M** are **structured node expressions**, representing more complex node structures, such as nodes with restricted scope or nodes running in parallel.

Transition rules for ω 0-calculus basic node expressions

- **MCast**: Describes broadcasting (sending) a message in a group **G**, provided that **G** is non-empty. This rule allows a process to broadcast a message to all nodes within its communication range.
- **RECV**: Describes receiving a message in a group **G**, provided that **G** is non-empty. This rule allows a process to receive any broadcasted message if it shares a group name with the sender.
- **CHOICE**: Describes non-deterministic choice between two processes **P** and **Q**. This rule allows nondeterministic behavior.
- **MATCH**: Matches values $x=x$ and proceeds with process **P**. Process Matching.
- **DEF**: Describes process invocation where parameters are substituted. Process Definition.

Transmission Range:

- Nodes can only send and receive messages if they share a common group name (i.e., are within each other's transmission range). A node with an empty interface (no group names) cannot send or receive messages, as it is effectively disconnected from the network.

Late Semantics:

- The communication model follows **late semantics**, meaning that the names (e.g.,

identifiers or addresses) of received messages are instantiated only when the message is actually received.

Transition rules for ω_0 -calculus structured node expressions

- **COM**: This rule governs communication between nodes. A broadcast sender will synchronize with any receiver that shares a group name, meaning the receiver is within the sender's transmission range.
- **GNAME-RES1** states that a restricted gname cannot occur in a transition label.
- **GNAME-RES2** states that when all gnames of a local-broadcast-send action are restricted, it becomes a τ -action.
- **MOBILITY**: The semantics of node movement is defined by the MOBILITY rule, which states that the process interface of node $P : G$ can change from G to G' whenever the node is in parallel with another node M .
- **PAR** rule indicates the interleaving semantics for actions of nodes in parallel, to make sure that the operation between multiple nodes will not have conflicts, which means in the same time, different nodes in the system can execute their own actions respectively.